ANGULAR LIBRARY · NPM

Sticky Notes Library

Rich-text notes for any page or record

sticky-notes-lib 1.0.0 · MIT license · Angular 16+

Shivam Paliwal
Solutions Engineer · paliwalshivam93.vns@gmail.com

Technical documentation · October 2026

Contents

1. Overview 4

At a glance . . . 4

Features . . . 4

2. How it works 5

User experience . . . 5

Data flow . . . 5

3. Installation 6

Requirements . . . 6

Install the packages . . . 6

Add the styles . . . 6

Register the providers . . . 6

4. Quick start 7

Step 1: point the library at your API . . . 7

Step 2: add the component to a screen . . . 8

Choosing page codes . . . 8

5. API reference 8

StickyNotesComponent . . . 8

RequestService . . . 9

StickynotesService . . . 10

Loading . . . 10

CommonMessageService . . . 10

noWhitespaceValidator . . . 11

modules (editor toolbar) . . . 11

6. Backend contract 11

Endpoints . . . 11

Note model . . . 12

Store the HTML safely . . . 12

7. Behaviour details 13

Validation . . . 13

Messages . . . 13

Labels . . . 13

8. Styling 14

9. Troubleshooting 15

10. Development and release 15

Version history . . . 15

1. Overview

Sticky Notes Library is an Angular library that adds rich-text notes to any screen of an application. One standalone component gives users a floating notes button and a side panel where they can write, format, update and delete a note attached to the record they are looking at.

Enterprise applications often need a lightweight way to keep context next to a record, such as a remark on a customer, a reviewer comment on an assessment, or a reminder on a plan. Building this separately for every screen means repeating the same form, editor, API calls and messages. The library packages the whole feature, so adding notes to a new screen is one line of template code.

At a glance

Item	Value
Package	sticky-notes-lib (npm)
Version	1.0.0, published 5 May 2026
License	MIT
Source	https://github.com/shivampaliwal24/sticky_notes_lib
Component selector	`<lib-sticky-notes-component>`
Component type	Standalone, so no NgModule is needed
Build format	Partial-Ivy FESM2022 bundle with TypeScript typings, side-effect free
Built with	Angular 21, compatible with Angular 16 and later
UI dependencies	PrimeNG (Sidebar, Editor, Button, Toast, Tooltip, InputText) and Quill 2

Features

- **Rich-text editing** with a full Quill toolbar: bold, italic, underline, strike, quotes, ordered and bullet lists, sub/superscript, indentation, right-to-left text, headings H1 to H6, text and background colours, fonts, alignment, links and clear formatting.
- **Complete note lifecycle:** create, load, update and delete, with the right REST call chosen automatically.
- **Context-aware:** each note is tied to a record ID and a page code, and the component reloads by itself when either one changes.
- **View-only mode** for users who may read notes but not change them.
- **Built-in validation:** a title is required, and the note must contain visible text, not just empty formatting.
- **User feedback:** toast messages for success and errors, plus a shared loading state your app can display.

- **Reusable building blocks:** the HTTP client, loading service, message service, validator and editor toolbar are all exported for use elsewhere.

2. How it works

User experience

1. A round **bookmark button** is fixed to the right edge of the screen, vertically centred. Hovering shows a "Notes" tooltip.
2. Clicking it opens a **side panel** from the right (40% of the viewport wide, non-modal, so the page stays usable). The panel header shows the title you pass to the component.
3. The panel contains a **Title** field and the **rich-text editor**. If a note already exists for this record, it is loaded into the form.
4. The footer shows **Save**, or **Update** when a note exists, and **Delete** when a note exists. In view-only mode both buttons are hidden and the fields are read-only.
5. After saving, a success toast appears and the panel stays open. After deleting, the form is cleared and the panel closes.

Data flow

Moment	What the component does	HTTP call
Component receives or changes `id` / `pageCode`	Resets the form and loads the note for the new context	`GET /api/notes`
User opens the panel	Checks that both id and pageCode are set, then reloads the note	`GET /api/notes`
User clicks Save (no existing note)	Validates, then creates the note	`POST /api/notes`
User clicks Update	Validates, then updates the existing note	`PUT /api/notes/{id}`
User clicks Delete	Deletes the note, clears the form, closes the panel	`DELETE /api/notes/{id}`

The component keeps **one note per record and page**. When notes are loaded, the first item in the response becomes the current note. A save counts as an update only when a note was loaded and its page code matches the current one. Otherwise a new note is created.

3. Installation

Requirements

Dependency	Version	Used for
Node.js	^16.14.0 or >=18.13.0	Build tooling
`@angular/core`, `common`, `forms`	>=16.0.0	Component, HTTP, reactive forms
`primeng`	>=15.0.0	Sidebar, Editor, Toast, Button, Tooltip, InputText
`primeicons`	>=6.0.0	Bookmark, save and delete icons
`quill`	^2.0.0	Rich-text editor engine
`rxjs`	^7.8.0	Observables and loading state

Install the packages

```
npm install sticky-notes-lib
npm install primeng primeicons quill
```

The Angular packages and RxJS are normally already in an Angular project.

Add the styles

PrimeNG, PrimeIcons and Quill ship their CSS separately. Add them to the `styles` array in `angular.json`. The PrimeNG theme path depends on your PrimeNG version. This example is for PrimeNG 15 to 17:

```
"styles": [
  "node_modules/primeng/resources/themes/lara-light-blue/theme.css",
  "node_modules/primeng/resources/primeng.min.css",
  "node_modules/primeicons/primeicons.css",
  "node_modules/quill/dist/quill.snow.css",
  "src/styles.css"
]
```

Register the providers

The library uses Angular's `HttpClient` and PrimeNG's `MessageService`. PrimeNG's side panel and toasts also need animations. Provide all three once, at application level:

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { provideAnimations } from '@angular/platform-browser/animations';
import { MessageService } from 'primeng/api';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [provideHttpClient(), provideAnimations(), MessageService]
});
```

PrimeNG version note. The component imports `SidebarModule` from `primeng/sidebar`. Newer PrimeNG releases replace Sidebar with a Drawer component, so check that your PrimeNG version still provides `primeng/sidebar` before upgrading.

4. Quick start

Step 1: point the library at your API

Every request goes through `RequestService`, which prefixes paths with a base URL. Set it once, ideally from your environment configuration:

```
import { Component, OnInit } from '@angular/core';
import { RequestService } from 'sticky-notes-lib';
import { environment } from '../environments/environment';

@Component({ selector: 'app-root', standalone: true, template: '<router-outlet />' })
export class AppComponent implements OnInit {
  constructor(private requestService: RequestService) {}

  ngOnInit(): void {
    this.requestService.setBaseUrl(environment.apiUrl);   // e.g. https://api.myapp.com
  }
}
```

Step 2: add the component to a screen

```
import { Component } from '@angular/core';
import { StickyNotesComponent } from 'sticky-notes-lib';

@Component({
  selector: 'app-customer-details',
  standalone: true,
  imports: [StickyNotesComponent],
  template: `
    <h1>Customer details</h1>

    <lib-sticky-notes-component
      [id]="customer.id"
      pageCode="CUSTOMER_DETAILS"
      title="Customer notes"
      [viewOnly]="!canEdit">
    </lib-sticky-notes-component>
  `
})
export class CustomerDetailsComponent {
  customer = { id: 12345 };
  canEdit = true;
}
```

That is all you need. The bookmark button appears on the right of the screen, and the note for customer 12345 on the `CUSTOMER_DETAILS` page is loaded automatically.

Choosing page codes

- Use one stable, upper-case code per screen or entity type, for example `CUSTOMER_DETAILS` or `BIA_ASSESSMENT`.
- The code is stored with every note, so do not rename codes after go-live or existing notes will stop appearing.
- Keep the codes in one shared constants file so screens cannot drift apart.

5. API reference

StickyNotesComponent

Selector `lib-sticky-notes-component`. Standalone. Inputs:

Input	Type	Default	Description
`id`	`number`	`undefined`	ID of the record the note belongs to. Required before the panel can open.
`pageCode`	`string`	`undefined`	Code of the page or context. Required before the panel can open.

Input	Type	Default	Description
`title`	`string`	`''`	Heading of the side panel, also used as the Title field placeholder.
`viewOnly`	`boolean`	`false`	Disables the form and hides Save and Delete.

Changing `id` or `pageCode` at runtime, for example when navigating between records on the same screen, resets the form and loads the note for the new record.

RequestService

A thin, injectable wrapper around `HttpClient`, provided in root. You can use it for your own calls too.

Method	Returns	Notes
`setBaseUrl(url: string)`	`void`	Prefix for every relative endpoint
`get<T>(endpoint, params?, options?)`	`Observable<HttpResponse<T[]>>`	Returns the full response (body, headers, status). Null or undefined params are dropped.
`post<T>(endpoint, data, options?)`	`Observable<T>`	JSON body
`put<T>(endpoint, data, options?)`	`Observable<T>`	JSON body
`patch<T>(endpoint, data, options?)`	`Observable<T>`	JSON body
`delete<T>(endpoint, options?)`	`Observable<T>`	
`request<T>(method, endpoint, data?, options?)`	`Observable<T>`	Generic form, method is GET, POST, PUT, DELETE or PATCH

- Endpoints that start with `http://` or `https://` are used as-is, ignoring the base URL.
- Errors are logged to the console and passed on unchanged, so your subscriber still receives the original `HttpErrorResponse`.
- `RequestOptions` accepts `headers`, `params`, `observe`, `responseType`, `reportProgress` and `withCredentials`, the same as Angular's HttpClient options.

```
this.requestService
  .get<Customer>('/api/customers', { 'name.contains': 'acme', page: 0 })
  .subscribe(res => {
    const customers = res.body ?? [];
    const total = res.headers.get('X-Total-Count');
  });
```

StickynotesService

Method	HTTP call
`getNotesByIdAndPageCode(id?, pageCode?)`	`GET /api/notes?id.equals={id}&pageCode.equals={pageCode}`
`addNotes(data)`	`POST /api/notes`
`updateNotes(id, data)`	`PUT /api/notes/{id}`
`deleteNotes(id)`	`DELETE /api/notes/{id}`

Use it directly if you want to show notes elsewhere, for example a "has notes" badge in a list.

Loading

A global loading flag. The component calls `show()` before each request and `hide()` when it completes or fails. Subscribe to it to display a spinner:

```
import { AsyncPipe, NgIf } from '@angular/common';
import { Loading } from 'sticky-notes-lib';

@Component({
  selector: 'app-spinner', standalone: true, imports: [NgIf, AsyncPipe],
  template: `<div class="overlay" *ngIf="loading.isLoading | async">Loading...</div>`
})
export class SpinnerComponent {
  constructor(public loading: Loading) {}
}
```

Member	Description
`isLoading: BehaviorSubject<boolean>`	Current state, starting at false
`show()` / `hide()`	Set the state to true or false

CommonMessageService

Shows PrimeNG toasts, which the component renders at the top right. It requires `MessageService` to be provided.

Method	Severity	Shown for
`success(action, message)`	success	3 seconds
`info(action, message)`	info	3 seconds
`warning(action, message)`	warn	4 seconds
`error(action, message)`	error	5 seconds
`clear()`		Removes all toasts

noWhitespaceValidator

A reactive-forms validator that rejects values made only of spaces. Empty values pass, so combine it with `Validators.required`. On failure it returns `{ nospace: true }`.

```
import { Validators } from '@angular/forms';
import { noWhitespaceValidator } from 'sticky-notes-lib';

name = new FormControl('', [Validators.required, noWhitespaceValidator]);
```

modules (editor toolbar)

The Quill toolbar configuration used by the component. Reuse it to give other editors in your app the same toolbar:

```
import { modules } from 'sticky-notes-lib';

// template:  <p-editor [modules]="modules" formControlName="description"></p-editor>
export class ArticleFormComponent {
  modules = modules;
}
```

6. Backend contract

The component talks to four REST endpoints under the configured base URL. The query syntax follows JHipster criteria filtering, so a JHipster application can provide them with a Note entity and filtering enabled.

Endpoints

Method	Path	Request body	Expected response
GET	`/api/notes?id.equals={id}&pageCode.equals={pageCode}`		JSON array of notes. The first element is used, and an empty array means no note yet.
POST	`/api/notes`	Note	The saved note
PUT	`/api/notes/{id}`	Note	The updated note
DELETE	`/api/notes/{id}`		Any 2xx status

Note model

Field	Type	Description
`id`	number	On create, the component sends the host record ID it was given. On update, it sends the ID of the loaded note.
`title`	string	Required
`notes`	string (HTML)	Editor content as HTML, for example `<p>Call back on <b>Monday</b></p>`
`pageCode`	string	The page code the note belongs to

```
POST /api/notes
{
  "id": 12345,
  "title": "Follow-up",
  "notes": "<p>Call back on <b>Monday</b></p>",
  "pageCode": "CUSTOMER_DETAILS"
}

GET /api/notes?id.equals=12345&pageCode.equals=CUSTOMER_DETAILS
[
  { "id": 12345, "title": "Follow-up", "notes": "<p>Call back on ...</p>",
    "pageCode": "CUSTOMER_DETAILS" }
]
```

> **Important for the backend.** The record ID is sent in the `id` field on create, and notes are looked up with `id.equals`. Your API must therefore accept a client-supplied `id` on POST. JHipster's generated resources reject that by default with "A new entity cannot already have an ID". Also make sure (id, pageCode) is what identifies a note, so that records with the same ID on different pages do not overwrite each other.

Store the HTML safely

- Use a large text column (`TEXT` / `CLOB`) for `notes`.
- Treat the HTML as user input. Sanitize it on the server, or render it only through Angular bindings, which sanitize `[innerHTML]`.
- Protect the endpoints with the same authorization as the record they annotate.

7. Behaviour details

Validation

- **Title** is required.
- **Notes** are required and must not be only spaces. Before saving, the HTML is also reduced to plain text, so content with formatting but no visible text, such as an empty paragraph, is rejected too.
- When validation fails, an error toast shows "Please fill all required fields" and nothing is sent.

Messages

Situation	Message
Panel opened without id or pageCode	ID and Page Code are required
Validation failed	Please fill all required fields
New note saved	Saved Successfully
Existing note updated	Record Updated
Note deleted	Deleted Successfully
Delete with no note loaded	No notes to delete
Save or delete failed	The server's `message` field, or "Failed to save notes" / "Failed to delete notes"

Labels

Button and message texts come from the component's public `generalTexts` object. Each entry has a `labelId` and falls back to an English default:

Key	labelId	Default text
`saveButton`	`L_SAVE`	Save
`recordUpdated`	`L_RECORD_UPDATED`	Record Updated
`savedMsg`	`L_SAVED`	Saved Successfully
`mandatory`	`L_MANDATORY_MSG`	Please fill all required fields
`notesButton`	`L_NOTES`	Notes (tooltip)
`titleLabel`	`L_TITLE`	Title
`deleteButton`	`L_DELETE`	Delete
`deleteConfirm`	`L_DELETE_CONFIRM`	Are you sure you want to delete?
`deletedMsg`	`L_DELETED`	Deleted Successfully

Defaults are applied only to entries without a `label`. To translate the texts, set the labels on the component instance, for example through `@ViewChild`, before the user opens the panel:

```
@ViewChild(StickyNotesComponent) notes!: StickyNotesComponent;

ngAfterViewInit(): void {
  this.notes.generalTexts.saveButton.label = 'Speichern';
  this.notes.generalTexts.deleteButton.label = 'Löschen';
}
```

The Update button text and the "Title" field label are currently fixed in the template.

8. Styling

The component ships with compact default styles. Override them from your global stylesheet:

Selector	Element	Default
`#show-notes`	Floating notes button	40 px circle, #365367, fixed to the right edge at 50% height, z-index 999, grows on hover
`#show-notes .pi-bookmark`	Button icon	White, 20 px
`.notes-content`	Panel body	Scrollable, 1rem padding
`.p-sidebar-right`	Side panel	85vh tall, starting 5vh from the top
`.fixed-editor .ql-editor`	Editor area	Fills the remaining height and scrolls

```
/* styles.css: match your brand and move the button lower */
#show-notes { background: #1f4e79; top: 70%; }
#show-notes .pi-bookmark { font-size: 18px; }
```

9. Troubleshooting

Symptom	Cause and fix
`NullInjectorError: No provider for HttpClient`	Add `provideHttpClient()` to the application providers.
`No provider for MessageService`	Add PrimeNG's `MessageService` to the providers.
Toast or panel does not animate, or animation errors	Add `provideAnimations()`.
Editor has no toolbar styling	Add `quill/dist/quill.snow.css` to the styles.
Icons show as empty squares	Add `primeicons/primeicons.css` to the styles.
"ID and Page Code are required"	The `id` or `pageCode` input is empty when the button is clicked. Check the bindings, for example data that has not loaded yet.
Requests go to the Angular dev server	The base URL was not set. Call `setBaseUrl()` at startup.
Create fails with HTTP 400	The backend rejects a POST that already has an `id`. See chapter 6.
CORS errors in the browser	Allow the app's origin on the API, or use an Angular dev-server proxy.

10. Development and release

```
npm run build:lib            # build into dist/sticky-notes-lib
npm run build:lib:watch      # rebuild on every change
ng test sticky-notes-lib     # unit tests
npm run pack:lib             # create a .tgz for local testing

cd dist/sticky-notes-lib
npm publish
```

Version history

Version	Date	Changes
1.0.0	5 May 2026	First public release

Released under the MIT license. Source: https://github.com/shivampaliwal24/sticky_notes_lib · Package: https://www.npmjs.com/package/sticky-notes-lib